The dataset: ecommerce_orders

One year (January to December 2024) of orders from an Indian online store: **300 orders** placed by **55 customers** in **12 cities**, across **6 categories** and **30 products**. Like real data, it has repeat customers, cancelled and returned orders, tied amounts (fixed prices) and a few orders that were accidentally entered twice.

Table structure

Column	Type	Meaning
`order_id`	INT	Unique order number
`order_date`	DATE	Date the order was placed (Jan to Dec 2024)
`customer_id`	VARCHAR	Customer code, e.g. C025
`customer_name`	VARCHAR	Customer's full name
`city`	VARCHAR	Delivery city
`state`	VARCHAR	State of the city
`category`	VARCHAR	Electronics, Fashion, Home & Kitchen, Beauty, Books, Sports
`product_name`	VARCHAR	Product bought (5 products per category)
`quantity`	INT	Units in the order (1 to 4)
`unit_price`	INT	Price of one unit in rupees
`discount_pct`	INT	Discount given: 0, 5, 10 or 20 %
`total_amount`	INT	quantity x unit_price x (100 - discount_pct) / 100, rounded
`payment_mode`	VARCHAR	UPI, Credit Card, Debit Card, COD, Net Banking
`order_status`	VARCHAR	Delivered, Cancelled or Returned

Revenue always means the sum of `total_amount` for orders with `order_status = 'Delivered'`. Step 1 of every question tells you which orders to use.

Sample rows (first 10 of the table)

order_id	order_date	customer_name	city	category	product_name	qty	price	disc	amount	status
1	2024-01-01	Isha Reddy	Lucknow	Fashion	Leather Wallet	2	699	0	1398	Delivered
2	2024-01-03	Sakshi Singh	Ahmedabad	Fashion	Denim Jeans	1	1499	0	1499	Delivered
3	2024-01-03	Shreya Nair	Chennai	Fashion	Cotton Kurta	1	899	5	854	Delivered
4	2024-01-04	Isha Reddy	Lucknow	Home & Kitchen	Air Fryer	1	5999	0	5999	Returned
5	2024-01-05	Karan Rao	Bengaluru	Home & Kitchen	Mixer Grinder	2	3499	5	6648	Delivered
6	2024-01-07	Swati Gupta	Kolkata	Electronics	Bluetooth Speaker	2	2499	5	4748	Delivered
7	2024-01-08	Saurabh Malhotra	Jaipur	Books	Data Analytics Guide	1	899	5	854	Delivered
8	2024-01-08	Anjali Mehta	Indore	Electronics	Bluetooth Speaker	2	2499	10	4498	Cancelled
9	2024-01-10	Mohit Banerjee	Ahmedabad	Electronics	Mechanical Keyboard	1	3999	20	3199	Delivered
10	2024-01-11	Anjali Mehta	Indore	Books	Python Crash Course	2	699	0	1398	Delivered

Section A : OVER()

An empty OVER () makes the window the WHOLE table. Every row keeps its own details and also sees the overall total, average or count.

Q1. Total number of orders on every row

Scenario: The manager wants to see each order together with the total number of orders placed in the year.

Steps

1. Use ALL orders (Delivered, Cancelled and Returned).
2. On every row, show the total number of orders using COUNT(*) OVER ().

Sort the result by: `order_id`

Hint: `COUNT(*) OVER ()`

Note: Unlike GROUP BY, a window function keeps every row. An empty OVER () means 'look at the whole table'.

Output table structure

order_id	customer_name	total_amount	total_orders
Order number	*Customer*	*Amount of this order*	*Number of orders in the table (same on every row)*
1	Isha Reddy	1398	300
2	Sakshi Singh	1499	300
3	Shreya Nair	854	300

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 297 more rows. **Expected rows: 300**

Q2. Flag high-value orders

Scenario: Marketing calls an order 'High Value' if it is bigger than the company's average order value (AOV).

Steps

1. Use only orders where order_status = 'Delivered'.
2. On every row, show the company AOV = average of total_amount over all delivered orders, rounded to 2 decimals.
3. Add order_type: 'High Value' if total_amount is greater than the AOV, otherwise 'Normal'.

Sort the result by: `order_id`

Hint: `ROUND(AVG(total_amount) OVER (), 2)` and `CASE WHEN total_amount > AVG(total_amount) OVER () THEN ... ELSE ... END`

Note: You can use a window function inside CASE WHEN, just like a normal column.

Output table structure

order_id	customer_name	total_amount	company_aov	order_type
Order number	*Customer*	*Amount of this order*	*Average order value (same on every row)*	*'High Value' or 'Normal'*
1	Isha Reddy	1398	2800.29	Normal
2	Sakshi Singh	1499	2800.29	Normal
3	Shreya Nair	854	2800.29	Normal

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Q3. Units ordered vs the whole year

Scenario: The warehouse team wants to compare each order's units with the whole year's volume.

Steps

1. Use ALL orders (Delivered, Cancelled and Returned).
2. On every row, show the total number of orders and the total units (sum of quantity) for the whole year.
3. Calculate pct_of_units = quantity x 100 / total_units, rounded to 2 decimals.

Sort the result by: `order_id`

Hint: `COUNT(*) OVER (), SUM(quantity) OVER (), and quantity * 100.0 / SUM(quantity) OVER ()`

Note: Multiply by 100.0 (not 100) so MySQL does not lose the decimals.

Output table structure

order_id	product_name	quantity	total_orders	total_units	pct_of_units
Order number	*Product*	*Units in this order*	*Number of orders in the year*	*Sum of quantity in the year*	*This order's % of total_units*
1	Leather Wallet	2	300	508	0.39
2	Denim Jeans	1	300	508	0.20
3	Cotton Kurta	1	300	508	0.20

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 297 more rows. **Expected rows: 300**

Section B : PARTITION BY

PARTITION BY splits the rows into groups (category, city, customer, month). The calculation restarts for each group, but no rows are removed.

Q4. Highest and lowest order of the year

Scenario: Finance wants every order shown next to the year's biggest and smallest order.

Steps

1. Use only orders where order_status = 'Delivered'.
2. On every row, show MAX(total_amount) and MIN(total_amount) over the whole table.

Sort the result by: `order_id`

Hint: `MAX(total_amount) OVER () and MIN(total_amount) OVER ()`

Note: The WHERE clause runs first, so the window only sees Delivered orders.

Output table structure

order_id	product_name	total_amount	highest_order	lowest_order
Order number	*Product*	*Amount of this order*	*Biggest order amount of the year*	*Smallest order amount of the year*
1	Leather Wallet	1398	15996	269
2	Denim Jeans	1499	15996	269
3	Cotton Kurta	854	15996	269

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Q5. Category revenue and share

Scenario: Each category manager wants to see every order next to the revenue of their own category.

Steps

1. Use only orders where order_status = 'Delivered'.
2. On every row, show the total revenue of that order's CATEGORY.
3. Calculate pct_of_category = total_amount x 100 / category_revenue, rounded to 2 decimals.

Sort the result by: `category, order_id`

Hint: `SUM(total_amount) OVER (PARTITION BY category)`

Note: PARTITION BY restarts the calculation for each category. The % values inside one category add up to 100.

Output table structure

order_id	category	product_name	total_amount	category_revenue	pct_of_category
Order number	*Category*	*Product*	*Amount of this order*	*Revenue of this category*	*This order's % of its category*
15	Beauty	Perfume	1998	35055	5.70
25	Beauty	Hair Dryer	1424	35055	4.06
26	Beauty	Sunscreen SPF 50	1996	35055	5.69

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Q6. City average next to each order

Scenario: City managers want to compare each order with the usual order size in their city.

Steps

1. Use only orders where order_status = 'Delivered'.
2. On every row, show the average total_amount of that order's CITY, rounded to 2 decimals.

Sort the result by: `city, order_id`

Hint: `ROUND(AVG(total_amount) OVER (PARTITION BY city), 2)`

Note: Every order of the same city shows the same city_avg.

Output table structure

order_id	city	customer_name	total_amount	city_avg
Order number	*City*	*Customer*	*Amount of this order*	*Average order value of the city*
2	Ahmedabad	Sakshi Singh	1499	2928.42
9	Ahmedabad	Mohit Banerjee	3199	2928.42
12	Ahmedabad	Sakshi Singh	2799	2928.42

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Q7. Customer lifetime value on every order

Scenario: The CRM team wants each order to also show how valuable that customer is overall.

Steps

1. Use only orders where order_status = 'Delivered'.
2. On every row, show how many delivered orders that CUSTOMER has.
3. On every row, show the customer's lifetime value = sum of their total_amount.

Sort the result by: `customer_id, order_id`

Hint: `COUNT(*) OVER (PARTITION BY customer_id) and SUM(total_amount) OVER (PARTITION BY customer_id)`

Note: Partition by customer_id, not customer_name: two customers can share the same name.

Output table structure

customer_id	customer_name	order_id	total_amount	customer_orders	lifetime_value
Customer code	*Customer*	*Order number*	*Amount of this order*	*Customer's number of orders*	*Customer's total spend*
C001	Harsh Verma	162	664	1	664
C003	Kriti Gupta	47	1999	4	23490
C003	Kriti Gupta	76	9996	4	23490

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Q8. Monthly revenue and share

Scenario: Management reviews revenue month by month and wants each order's share of its month.

Steps

1. Use only orders where order_status = 'Delivered'.
2. Create order_month in 'YYYY-MM' format, e.g. 2024-01 (MySQL: DATE_FORMAT(order_date, '%Y-%m')).
3. On every row, show the total revenue of that MONTH.
4. Calculate pct_of_month = total_amount x 100 / month_revenue, rounded to 2 decimals.

Sort the result by: `order_month, order_id`

Hint: `DATE_FORMAT(order_date, '%Y-%m') AS order_month, then PARTITION BY DATE_FORMAT(order_date, '%Y-%m')`

Note: You cannot use the alias order_month inside OVER (). Repeat the full expression.

Output table structure

order_month	order_id	total_amount	month_revenue	pct_of_month
Month, e.g. 2024-01	*Order number*	*Amount of this order*	*Revenue of this month*	*This order's % of the month*
2024-01	1	1398	85206	1.64
2024-01	2	1499	85206	1.76
2024-01	3	854	85206	1.00

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Section C : ROW_NUMBER()

ROW_NUMBER() numbers rows 1, 2, 3 ... in the order you choose. Every row gets a different number, even when values tie.

Q9. Highest order of each category

Scenario: Category managers want to see how each order compares with the biggest order in their category.

Steps

1. Use only orders where order_status = 'Delivered'.
2. On every row, show MAX(total_amount) of that order's CATEGORY.

Sort the result by: `category, order_id`

Hint: `MAX(total_amount) OVER (PARTITION BY category)`

Note: Orders where total_amount = category_max are the biggest orders of their category.

Output table structure

order_id	category	product_name	total_amount	category_max
Order number	*Category*	*Product*	*Amount of this order*	*Biggest order amount in the category*
15	Beauty	Perfume	1998	4047
25	Beauty	Hair Dryer	1424	4047
26	Beauty	Sunscreen SPF 50	1996	4047

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Q10. Purchase sequence of each customer

Scenario: The CRM team wants to know whether an order was a customer's 1st, 2nd, 3rd ... purchase.

Steps

1. Use ALL orders (Delivered, Cancelled and Returned).
2. Number each CUSTOMER's orders from 1, oldest order first.
3. If two orders of a customer have the same date, the lower order_id comes first.
4. The numbering must restart at 1 for every customer.

Sort the result by: `customer_id, purchase_no`

Hint: `ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date, order_id)`

Note: ROW_NUMBER needs ORDER BY inside OVER (). PARTITION BY makes the numbering restart for each customer.

Output table structure

customer_id	customer_name	order_id	order_date	purchase_no
Customer code	*Customer*	*Order number*	*Order date*	*1 = first purchase, 2 = second ...*
C001	Harsh Verma	30	2024-02-04	1
C001	Harsh Verma	162	2024-07-23	2
C003	Kriti Gupta	47	2024-02-17	1

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 297 more rows. **Expected rows: 300**

Q11. Serial number for every order

Scenario: The operations team wants a running serial number for all orders of the year.

Steps

1. Use ALL orders (Delivered, Cancelled and Returned).
2. Number all orders from 1, oldest order first, using ROW_NUMBER().
3. If two orders have the same date, the lower order_id comes first.

Sort the result by: `serial_no`

Hint: `ROW_NUMBER() OVER (ORDER BY order_date, order_id)`

Note: Without order_id as a tie-breaker, orders on the same date could get different numbers each time you run the query.

Output table structure

order_id	order_date	customer_name	serial_no
Order number	*Order date*	*Customer*	*1 = oldest order of the year*
1	2024-01-01	Isha Reddy	1
2	2024-01-03	Sakshi Singh	2
3	2024-01-03	Shreya Nair	3

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 297 more rows. **Expected rows: 300**

Q12. Newest orders first in each city

Scenario: Each city's delivery team wants its orders numbered from the most recent one.

Steps

1. Use ALL orders (Delivered, Cancelled and Returned).
2. Number the orders inside each CITY, newest date = 1, using ROW_NUMBER().
3. If two orders have the same date, the higher order_id comes first.

Sort the result by: `city, city_order_no`

Hint: `ROW_NUMBER() OVER (PARTITION BY city ORDER BY order_date DESC, order_id DESC)`

Note: DESC puts the newest order first, so it gets number 1.

Output table structure

city	order_id	order_date	city_order_no
City	*Order number*	*Order date*	*1 = newest order of the city*
Ahmedabad	299	2024-12-29	1
Ahmedabad	292	2024-12-18	2
Ahmedabad	277	2024-12-02	3

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 297 more rows. **Expected rows: 300**

Q13. Number orders by amount in each category

Scenario: Category managers want their orders listed with a position number from the biggest to the smallest.

Steps

1. Use only orders where order_status = 'Delivered'.
2. Number the orders inside each CATEGORY, biggest total_amount = 1, using ROW_NUMBER().
3. If two orders have the same amount, the lower order_id comes first.

Sort the result by: `category, position_in_category`

Hint: `ROW_NUMBER() OVER (PARTITION BY category ORDER BY total_amount DESC, order_id)`

Note: Tied amounts still get different numbers with ROW_NUMBER. That is the difference from RANK.

Output table structure

category	order_id	total_amount	position_in_category
Category	*Order number*	*Amount of this order*	*1 = biggest order of the category*
Beauty	37	4047	1
Beauty	282	2998	2
Beauty	212	2848	3

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Section D : RANK()

RANK() gives tied values the same rank and then SKIPS numbers (1, 1, 3).

Q14. Product leaderboard inside each category

Scenario: Category managers want to know which of their products earns the most.

Steps

1. Use only orders where order_status = 'Delivered'.
2. Calculate the total revenue of each PRODUCT (one row per product).
3. Rank the products inside their CATEGORY, highest revenue = 1, using RANK().

Sort the result by: `category, rank_in_category`

Hint: `GROUP BY category, product_name, then RANK() OVER (PARTITION BY category ORDER BY SUM(total_amount) DESC)`

Note: A window function can use an aggregate like SUM(total_amount). GROUP BY runs first, then the window.

Output table structure

category	product_name	product_revenue	rank_in_category
Category	*Product*	*Total revenue of the product*	*1 = best product of the category*
Beauty	Hair Dryer	14240	1
Beauty	Perfume	8641	2
Beauty	Sunscreen SPF 50	5788	3

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 27 more rows. **Expected rows: 30**

Q15. Rank all orders by amount

Scenario: Finance wants a ranking of every delivered order by its amount.

Steps

1. Use only orders where order_status = 'Delivered'.
2. Rank the orders by total_amount, highest = 1, using RANK(). Tied amounts share a rank.

Sort the result by: `amount_rank, order_id`

Hint: `RANK() OVER (ORDER BY total_amount DESC)`

Note: Tied amounts share a rank and the next rank is skipped: 1, 2, 2, 4.

Output table structure

order_id	customer_name	total_amount	amount_rank
Order number	*Customer*	*Amount of this order*	*1 = biggest order*
20	Bhavna Verma	15996	1
23	Shreya Verma	12596	2
90	Kavya Iyer	11998	3

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Q16. Rank orders inside each city

Scenario: Each city manager wants their own ranking of orders by amount.

Steps

1. Use only orders where order_status = 'Delivered'.
2. Rank the orders inside each CITY by total_amount, highest = 1, using RANK().

Sort the result by: `city, city_rank, order_id`

Hint: `RANK() OVER (PARTITION BY city ORDER BY total_amount DESC)`

Note: The rank starts again at 1 for every city.

Output table structure

city	order_id	total_amount	city_rank
City	*Order number*	*Amount of this order*	*1 = biggest order of the city*
Ahmedabad	20	15996	1
Ahmedabad	205	8996	2
Ahmedabad	40	7598	3

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Section E : DENSE_RANK()

DENSE_RANK() gives tied values the same rank and does NOT skip (1, 1, 2).

Q17. Rank customers by total spend

Scenario: The company wants a leaderboard of customers by how much they spent in the year.

Steps

1. Use only orders where order_status = 'Delivered'.
2. Calculate each customer's total spend with GROUP BY (one row per customer).
3. Rank the customers by total spend, highest = 1, using RANK().

Sort the result by: `spend_rank, customer_id`

Hint: `GROUP BY customer_id, customer_name, then RANK() OVER (ORDER BY SUM(total_amount) DESC)`

Note: One row per customer. Customers with the same total share a rank.

Output table structure

customer_id	customer_name	total_spent	spend_rank
Customer code	*Customer*	*Total spend in the year*	*1 = biggest spender*
C012	Saurabh Pillai	50246	1
C043	Sakshi Singh	49099	2
C050	Bhavna Joshi	41375	3

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 49 more rows. **Expected rows: 52**

Q18. Dense rank of orders inside each category

Scenario: Category managers want order levels without gaps in the rank numbers.

Steps

1. Use only orders where order_status = 'Delivered'.
2. Rank the orders inside each CATEGORY by total_amount, highest = 1, using DENSE_RANK().

Sort the result by: `category, amount_level, order_id`

Hint: `DENSE_RANK() OVER (PARTITION BY category ORDER BY total_amount DESC)`

Note: DENSE_RANK has no gaps: 1, 2, 2, 3. Level 2 is the second highest different amount.

Output table structure

category	order_id	total_amount	amount_level
Category	*Order number*	*Amount of this order*	*1 = highest amount level of the category*
Beauty	37	4047	1
Beauty	282	2998	2
Beauty	212	2848	3

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**

Q19. City ranking inside each state

Scenario: State managers want to know which of their cities brings in the most revenue.

Steps

1. Use only orders where order_status = 'Delivered'.
2. Calculate the revenue of each CITY (one row per city).
3. On every row, also show the revenue of the city's STATE.
4. Rank the cities inside their state by revenue, highest = 1, using DENSE_RANK().

Sort the result by: `state, city_rank_in_state`

Hint: `GROUP BY state, city, then SUM(SUM(total_amount)) OVER (PARTITION BY state) and DENSE_RANK() OVER (PARTITION BY state ORDER BY SUM(total_amount) DESC)`

Note: SUM(SUM(...)) means: add up the city totals inside each state.

Output table structure

state	city	city_revenue	state_revenue	city_rank_in_state
State	*City*	*Revenue of the city*	*Revenue of the whole state*	*1 = top city of the state*
Delhi	Delhi	21061	21061	1
Gujarat	Ahmedabad	131779	131779	1
Karnataka	Bengaluru	124989	133184	1

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 9 more rows. **Expected rows: 12**

Q20. ROW_NUMBER vs RANK vs DENSE_RANK

Scenario: Compare the three ranking functions on the same data to see how they treat tied amounts.

Steps

1. Use only orders where order_status = 'Delivered'.
2. Order by total_amount, highest first, in all three functions.
3. For ROW_NUMBER, add order_id as a tie-breaker so the numbering is always the same.

Sort the result by: `row_num`

Hint: `Three window functions in one SELECT, all with ORDER BY total_amount DESC`

Note: Look at tied amounts: ROW_NUMBER 1, 2, 3 | RANK 1, 1, 3 | DENSE_RANK 1, 1, 2.

Output table structure

order_id	total_amount	row_num	rnk	dense_rnk
Order number	*Amount of this order*	*ROW_NUMBER: always unique*	*RANK: ties share, then skips*	*DENSE_RANK: ties share, no gaps*
20	15996	1	1	1
23	12596	2	2	2
90	11998	3	3	3

Row 2 explains each column. Rows below it are the first 3 rows of the correct answer ... and 255 more rows. **Expected rows: 258**